

UD3. Creando la primera aplicación

Contenido

1. Configurando tu Proyecto	2
2. Ejecutar una aplicación en Electron	4
3. Cargar una html en un <i>BrowserWindow</i>	5
3.1. Importando módulos	6
3.2. Escribir una función reutilizable para instanciar ventanas	6
3.3. Llamar a su función cuando la aplicación esté lista	6
4. Administrar el ciclo de vida de la ventana de la aplicación.....	7
4.1. Salir de la aplicación cuando todas las ventanas estén cerradas (<i>Windows y Linux</i>)	7
4.2. Abra una ventana si no hay ninguna abierta (<i>macOS</i>)	8
5. Recarga automática de electron	9
6. Creación de ejecutables con electron	9

Objetivos

Electron es un *framework* para crear aplicaciones de escritorio usando **JavaScript**, **HTML** y **CSS**. Incorpora [Chromium](#) como interfaz y [Node.js](#) en el lado servidor. Permite crear aplicaciones multiplataforma que funcionan en *Windows*, *macOS* y *Linux* con una única base común de código *JavaScript*. Por tanto, *Electron* es una capa de envoltorio nativa para aplicaciones web y se ejecuta en un entorno Node.js.

En esta parte se enseñará a configurar tu proyecto *Electron* y escribir una aplicación inicial sencilla. Al final de esta sección, podrás ejecutar una aplicación *Electron* en modo de desarrollo.

1. Configurando tu Proyecto

Las aplicaciones *Electron* se organizan usando **npm**, con el archivo *package.json* como punto de entrada. Comienza creando una carpeta e inicializando un paquete **npm** dentro de ella con el comando `npm init`.

```
$ mkdir mi-app-electron
$ cd mi-app-electron
$ npm init
```

Este comando pedirá que configures algunos atributos del *package.json*. Hay algunas reglas que seguir en esta guía:

- El punto de entrada de la aplicación debe ser el archivo *main.js*.
- *"author"*, *"license"* y *"description"* pueden tener cualquier valor, pero serán necesarios para el empaquetado más adelante.
- Se puede aceptar la configuración por defecto, normalmente indicada entre paréntesis, pulsando la tecla *intro*.

Ejemplo del proceso de configuración de `package.json` con `npm init`

This utility will walk you through creating a *package.json* file.
It only covers the most common items, and tries to guess sensible defaults.

Press `^C` at any time to quit.

package name: (my-electron-app)

version: (1.0.0)

description:

entry point: (index.js) main.js

test command:

git repository:

keywords:

author:

license: (ISC)

About to write to .package.json:

```
{
  "name": "my-electron-app",
  "version": "1.0.0",
  "description": "",
  "main": "main.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "author": "",
  "license": "ISC"
}
```

Is this OK? (yes)

Ahora, instala *Electron* en las *devDependencies* de la aplicación, que es la lista de dependencias de paquetes externos de desarrollo exclusivo que no se requieren en producción.

```
$ npm install electron --save-dev
```

¿Por qué Electron es una dependencia?

Esto puede parecer poco intuitivo ya que el código de producción ejecuta API de Electron. Sin embargo, las aplicaciones finales incluirán el binario de *Electron*, eliminando la necesidad de especificarlo como una dependencia de la aplicación binaria final a distribuir en producción.

El archivo package.json debería verse así después de inicializar su paquete e instalar *Electron*.

package.json

```
{
  "name": "my-electron-app",
  "version": "1.0.0",
  "description": "",
  "main": "main.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "author": "",
  "license": "ISC",
  "devDependencies": {
    "electron": "^26.2.2"
  }
}
```

También debe aparecer la carpeta `node_modules/` que contiene el ejecutable de *Electron*, así como un archivo de bloqueo `package-lock.json` que especifica las versiones de dependencia exactas que se instalarán.

2. Ejecutar una aplicación en Electron

El script `main` que definiste en `package.json` es el punto de inicio de cualquier aplicación. Este script controla el proceso principal, que se ejecuta en un entorno *Node.js* y es responsable de controlar el ciclo de vida de su aplicación, mostrar interfaces nativas, realizar operaciones con privilegios y administrar los procesos de renderizado.

Antes de crear la primera aplicación, primero utilizará un script para asegurar de que el punto de entrada del proceso principal está configurado correctamente. Crea un archivo `main.js` en la carpeta raíz del proyecto con una sola línea de código:

```
main.js
console.log('Hola desde Electron')
```

Debido a que el proceso principal es un intérprete de *Node.js*, se puede ejecutar código con el comando `electron`. Para ejecutar este script, agrega `"electron ."` al comando de inicio (`"start"`) en el atributo `"scripts"` del archivo `package.json`. Este comando le indicará al ejecutable que busque el script principal en el directorio actual y lo ejecute en modo de desarrollo.

`package.json`

```
{
  "name": "my-electron-app",
  "version": "1.0.0",
  "description": "Hello World!",
  "main": "main.js",
  "scripts": {
    "start": "electron .",
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "author": "Bea Fuster",
  "license": "ISC",
  "devDependencies": {
    "electron": "^38.1.2"
  }
}
```

```
$ npm start
```

El terminal debería mostrar `"Hola desde Electron."` ¡Enhorabuena, has ejecutado tu primera línea de código en Electron! A continuación, veremos cómo crear interfaces de usuario en HTML y cargarlas en una ventana nativa.

3. Cargar una html en un *BrowserWindow*

Cada ventana muestra una página web que se puede cargar desde un archivo **HTML** local o desde una dirección web remota. Para este ejemplo, se cargará desde un archivo local. Comienza creando una página web básica en un archivo **index.html** en la carpeta raíz del proyecto:

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Hola desde electron</title>
  </head>
  <body>
    <h1>Hola desde electron</h1>
  </body>
</html>
```

Ahora que tenemos una página web, se puede cargar en una ventana del navegador Electron. Reemplaza el contenido del archivo **main.js** con el siguiente código. Se explicará cada bloque por separado.

```
const { app, BrowserWindow } = require('electron');

const createWindow = () => {
  const win = new BrowserWindow({ width: 800, height: 600 });
  win.loadFile('index.html');
};

app.whenReady().then(() => {
  createWindow();
});
```

Si ejecutamos con `npm start` obtendremos nuestra primera aplicación con electron:



Para abrir la consola en la ventana de electron pulsar **Ctrl+Shift+i**
Y para recargar la página **Ctrl+r**

3.1. Importando módulos

```
const { app, BrowserWindow } = require('electron');
```

En la primera línea, estamos importando dos módulos Electron con la sintaxis del módulo *CommonJS*:

- `app`, controla el ciclo de vida del evento de su aplicación.
- `BrowserWindow`, que crea y administra ventanas de aplicaciones.

Convenciones de capitalización de nombres

Es posible que hayas notado la diferencia de mayúsculas entre `app` y los módulos de `BrowserWindow`. `Electron` sigue las convenciones típicas de *JavaScript*, donde los módulos se indican en **PascalCase** y son constructores de clases instanciables (por ejemplo, `BrowserWindow`, `Tray`, `Notification`) mientras que los módulos **camelCase** no son instanciables (por ejemplo, `app`, `ipcRenderer`, `webContents`).

3.2. Escribir una función reutilizable para instanciar ventanas

La función `createWindow()` carga la página web en una nueva instancia de `BrowserWindow`:

```
const createWindow = () => {  
  const win = new BrowserWindow({ width: 800, height: 600 });  
  win.loadFile('index.html');  
};
```

3.3. Llamar a su función cuando la aplicación esté lista

```
app.whenReady().then(() => {  
  createWindow();  
});
```

Muchos de los módulos principales son emisores de eventos de *Node.js* que se adhieren a la arquitectura asincrónica basada en eventos de Node. El módulo de aplicación es uno de estos emisores.

`BrowserWindows` solo se puede crear después de que se dispare el evento `ready` de la `app`. Puede esperar este evento utilizando la API `app.whenReady()` y llamando a `createWindow()` una vez que se cumpla su promesa.

 **evento** ready

Normalmente los eventos de *Node.js* se escuchan utilizando la función `.on` de un emisor.

```
app.on('ready', () => {  
  // app.whenReady().then(() => {  
    createWindow()  
  })  
})
```

Sin embargo, *Electron* expone `app.whenReady()` para evitar problemas al escuchar directamente el evento `ready`.

En este momento, al ejecutar el comando de start de la aplicación debería abrir con una ventana que muestre la página web.

Cada página web que la aplicación muestra en una ventana se ejecutará en un proceso separado llamado **renderer process** (proceso de renderizado) (o simplemente **renderer** para abreviar). Los procesos de renderizado tienen acceso a las mismas API de JavaScript y herramientas que se utilizan para el desarrollo web *front-end* típico, como el uso de **webpack** para agrupar y minimizar su código o **React** para crear interfaces de usuario.

4. Administrar el ciclo de vida de la ventana de la aplicación

Las ventanas de las aplicaciones se comportan de manera diferente en cada sistema operativo. En lugar de hacer cumplir estas convenciones de forma predeterminada, *Electron* ofrece la opción de implementarlas en el código de su aplicación si desea seguirlas. Puede implementar convenciones básicas de ventana escuchando los eventos emitidos por la aplicación y los módulos de `BrowserWindow`.

 **Flujo de control específico**

La verificación con la variable `process.platform` de *Node* puede ayudar a ejecutar código de manera condicional en ciertas plataformas. Se debe tener en cuenta que sólo hay tres plataformas posibles en las que *Electron* puede ejecutarse: `win32` (Windows), `linux` (Linux) y `darwin` (macOS).

4.1. Salir de la aplicación cuando todas las ventanas estén cerradas (*Windows* y *Linux*)

En *Windows* y *Linux*, cerrar todas las ventanas generalmente cerrará una aplicación por completo. Para implementar este patrón en *Electron*, se debe escuchar el evento `window-all-closed` del módulo de la aplicación y llama a `app.quit()` para salir de la aplicación si el usuario no está en *macOS*.

```
app.on('window-all-closed', () => {
  if (process.platform !== 'darwin') app.quit();
});
```

4.2. Abra una ventana si no hay ninguna abierta (*macOS*)

Por el contrario, el comportamiento por defecto de las aplicaciones de *macOS* es que generalmente continúan ejecutándose incluso sin ninguna ventana abierta. Activar la aplicación cuando no hay ventanas disponibles debería abrir una nueva.

Para implementar esta característica, escuchar el evento `activate` del módulo de la aplicación y llamar a su método `createWindow()` existente si no hay ningún `BrowserWindows` abierto.

Debido a que las ventanas no se pueden crear antes del evento `ready`, solo debería escuchar el evento `activate` después de inicializar la aplicación. Haz esto escuchando solo los eventos de activación dentro del callback de `whenReady()`.

```
app.whenReady().then(() => {
  createWindow();
});

app.on('activate', () => {
  if (BrowserWindow.getAllWindows().length === 0) createWindow();
});
```

Código final

```
const { app, BrowserWindow } = require('electron');

const createWindow = () => {
  const win = new BrowserWindow({ width: 800, height: 600 });
  win.loadFile('index.html');
};

app.whenReady().then(() => {
  createWindow();
});

app.on('activate', () => {
  if (BrowserWindow.getAllWindows().length === 0) createWindow();
});

app.on('window-all-closed', () => {
  if (process.platform !== 'darwin') app.quit();
});
```

5. Recarga automática de electron

Para evitar iniciar Electron cada vez que queramos probar algún cambio en nuestro código, podemos usar el paquete Electron-Reloader como dependencia para nuestro proyecto de desarrollo.

Terminal:

```
$ npm install --save-dev electron-reloader
```

Después, agrega esta instrucción try-catch en tu archivo main.js:

```
try {
  require('electron-reloader')(module, {
    debug: true,
    watchRenderer: true,
  });
} catch (_) {}
```

A partir de ahora, cuando iniciemos la aplicación con npm start, la aplicación se actualizará cada vez que cambiemos nuestro código.

6. Creación de ejecutables con electron

Para generar un ejecutable utilizaremos electron-packager, instalándolo como dependencia de desarrollo:

```
npm install electron-packager --save-dev
```

Además de instalar el módulo, se crea una entrada en package.json:

```
"electron-packager": "^ 17.1.2"
```

Ahora incluiremos una entrada para ejecutar este paquete desde npm:

```
"scripts": {
  "start": "electron .",
  "build": "electron-packager ."
},
```

Lo ejecutamos con:

```
npm run build
```

Esto creará un directorio dependiendo de la arquitectura y el sistema operativo, en mi caso windows: minimal-repro-win32-x64

Dentro podemos encontrar el ejecutable: minimal-repro.exe

Si queremos crear un ejecutable para otro sistema operativo como Linux, deberemos ejecutar:

npm run build -- --platform=linux

Fíjate que al usar el build los parámetros tenemos que pasarlos con -- escrito dos veces. Esto generará un directorio minimal-repro-linux-x64 con el ejecutable minimal-repro.

Si quieres que directamente cree un solo archivo ejecutable con todo en su interior puedes utilizar electron-builder, pero para que funcione deberás activar el modo desarrollador.